

Vextor7 - Solana-Based Stablecoin Wallet & Smart Contract Payment System

1. Project Overview

Vextor7 is a comprehensive multi-chain, non-custodial cryptocurrency wallet and payment system with a primary focus on the Solana blockchain (targeting 80% of traffic, Stage 1). It allows users to manage their crypto assets across various chains (Solana, Base, Polygon, Ethereum, Bitcoin - Stage 1), perform on-chain transactions, utilize an off-chain exchange for instant internal transfers (Stage 5), purchase stablecoins via fiat on-ramps (Stage 5), and make payments using QR codes, NFC (Stage 8), and invoicing (Stage 6). The system includes dedicated features for merchants (POS, dashboard, settlement - Stage 9) and adheres to regulatory compliance standards (Stage 10). Integration with DeFi/DEX (Stage 11) and NFT features (Stage 12) are also included.

Key Features (Mapped to Contract Stages):

- Multi-Chain Non-Custodial Wallet (Solana, Base, Polygon, ETH, BTC) [Stage 1]
- Trust Wallet Core Integration [Stage 1]
- Multi-Portfolio Management [Stage 1]
- Smart Contract-Based Payments (Solana, Base, Polygon) [Stage 2]
- Automatic Token Swaps (Jupiter, 1inch/Uniswap) [Stage 2, Stage 11]
- Off-Chain Internal Exchange & Ledger [Stage 5]
- Fiat On-Ramp Integration (Transak/Ramp) [Stage 5]
- QR & NFC Payment System [Stage 8]
- Invoice & Recurring Subscription System [Stage 6]
- Pull Payment Authorizations [Stage 6]
- Staking (USDC/USDT on Solana) & VIP Tiers [Stage 7]
- Merchant POS System & Dashboard [Stage 9]
- Regulatory Compliance (Travel Rule, Chainalysis Integration) [Stage 10]
- DeFi/DEX Integration (WalletConnect v2, Jupiter, Uniswap, etc.) [Stage 11]
- NFT Wallet Features (Solana & EVM) [Stage 12]
- Advanced Security (PIN/Biometrics, TOTP 2FA, KYC, Address Whitelisting) [Stage 3, Stage 18]
- Market Analysis Tools & Price Alerts [Stage 14]
- Privacy Features (Optional TOR, Address Rotation) [Stage 13]
- Customer Support Features (FAQ, Tickets) [Stage 17]
- Performance Optimization [Stage 16]
- Comprehensive UI/UX Overhaul [Stage 4]

- Notifications (Push & Email) [Stage 15]

2. Tech Stack

- **Backend:** FastAPI (Python 3.9+)
- **Database:** PostgreSQL (with asyncpg for async support)
- **Caching/Session/Nonce Mgmt:** Redis (with redis-py async)
- **Background Tasks:** Celery with Redis/RabbitMQ broker (Recommended) or FastAPI BackgroundTasks
- **Scheduling:** APScheduler (async version)
- **Blockchain Interaction:**
 - Solana: solana-py (async), Anchor (anchorpy if needed), Jupiter Aggregator API/SDK
 - EVM (Base, Polygon, Ethereum): web3.py (async), Solidity (Smart Contracts), 1inch API/SDK
 - Bitcoin: python-bitcoinlib or similar for SPV/transaction building
- **Frontend:** React Native (Expo recommended)
- **Wallet Core:** Trust Wallet Core (Integrated as native module)
- **Infrastructure:** AWS (ECS/Fargate or EC2 ASG, RDS Aurora Serverless/PostgreSQL, ElastiCache Redis, S3, KMS, ALB, CloudFront, CloudWatch, Route 53, ACM, VPC)
- **CI/CD:** GitHub Actions
- **Notifications:** Firebase Cloud Messaging (FCM), SendGrid or AWS SES
- **External Services:**
 - Fiat On-Ramp: Transak, Ramp Network APIs/SDKs
 - Market Data: CoinGecko API, CoinMarketCap API
 - Blockchain Analytics: Chainalysis API
 - DEX Aggregation: Jupiter Aggregator (Solana), 1inch (EVM)
 - Error Tracking: Sentry SDK
 - KYC Provider: (To be selected - e.g., Sumsb, Veriff API)
 - SMS Provider: Twilio API or similar
- **Smart Contract Dev:**
 - Solana: Anchor Framework (Rust)
 - EVM: Solidity, Hardhat/Foundry, OpenZeppelin Contracts

3. Project File Structure & File Descriptions

3.1. Backend (/vextor7-backend/)

/vextor7-backend/

```
|— .env           # Environment variables (DB URL, API Keys, JWT Secret, RPC URLs - NOT
COMMITTED)
|— .gitignore     # Specifies intentionally untracked files that Git should ignore
|— Dockerfile     # Instructions to build the backend Docker image for deployment
|— docker-compose.yml # Defines multi-container Docker applications (backend, db,
redis, celery, flower) for local dev
```

- ├── alembic.ini # Configuration file for Alembic database migrations
- ├── poetry.lock # Exact versions of dependencies (generated by Poetry)
- ├── pyproject.toml # Project metadata and Python dependencies (for Poetry)
- ├── README.md # This file
-
- ├── app/ # Main application source code package
 - ├── __init__.py # Makes 'app' a Python package
 - ├── main.py # Creates FastAPI app instance, includes root routers, sets up
middleware (CORS, Rate Limiting), exception handlers
 -
 - ├── api/ # API Routers, versioning, and endpoint definitions
 - ├── __init__.py # Makes 'api' a Python package
 - ├── v1/ # API Version 1 directory
 - ├── __init__.py # Imports and combines all v1 endpoint routers into a single
APIRouter for inclusion in main.py
 - ├── endpoints/ # Contains individual API endpoint modules (FastAPI Routers)
 - ├── __init__.py # Makes 'endpoints' a Python package
 - ├── auth.py # Defines /auth endpoints: POST /register, POST /login, POST
/refresh, POST /logout
 - ├── users.py # Defines /users endpoints: GET /me, PUT /me, GET /search
 - ├── security.py # Defines /security endpoints: POST /pin/change, POST
/otp/enable, POST /otp/verify, POST /otp/disable, GET /otp/backup-codes, POST
/kyc/submit, GET /kyc/status, POST /whitelist/add, GET /whitelist, DELETE
/whitelist/{address_id}
 - ├── wallets.py # Defines /wallets endpoints: POST /portfolios, GET /me, POST
/wallets, DELETE /wallets/{wallet_id}, PUT /portfolios/{portfolio_id}, GET /{address}/balance,
GET /{address}/history
 - ├── transactions.py # Defines /transactions endpoints: POST /prepare, POST
/broadcast, GET /status/{tx_hash}, POST /prepare/cancel
 - ├── exchange.py # Defines /exchange endpoints: POST /webhooks/onramp
(webhook), GET /balance, POST /internal_transfer, GET /internal_transfers, POST
/deposit/address, GET /deposits, POST /withdraw, GET /withdrawals
 - ├── payments.py # Defines /payments endpoints: POST /session, GET
/session/{session_id}, POST /execute/{session_id}
 - ├── invoices.py # Defines /invoices endpoints: POST /, GET /, GET
/{invoice_id}, POST /{invoice_id}/pay
 - ├── subscriptions.py # Defines /subscriptions & /pull-payments endpoints: POST
/subscriptions, GET /subscriptions, PUT /subscriptions/{subscription_id}, POST
/pull-payments/approve, GET /pull-payments/approvals, DELETE
/pull-payments/approvals/{approval_id}
 - ├── merchants.py # Defines /merchants endpoints: POST /register, GET /me,
POST /kyc, GET /dashboard, GET /transactions, POST /settlement, GET /settlements, POST
/employees, GET /employees

```

| | | | | — market.py    # Defines /market endpoints: GET /coins, GET
/coins/{coin_id}/chart, POST /alerts, GET /alerts, DELETE /alerts/{alert_id}
| | | | | — staking.py    # Defines /staking endpoints: GET /options, POST /stake, POST
/unstake, GET /positions, GET /history
| | | | | — vip.py        # Defines /vip endpoints: GET /status, GET /benefits
| | | | | — nfts.py        # Defines /nfts endpoints: GET /, POST /prepare-transfer
(Placeholder for potential future NFT backend logic)
| | | | | — defi.py        # Defines /defi endpoints: POST /swap/quote, POST
/swap/prepare
| | | | | — admin.py       # Defines /admin endpoints (protected): GET /users, PUT
/users/{user_id}, GET /kyc/submissions, PUT /kyc/submissions/{submission_id}, etc.
| | | | — deps.py         # FastAPI dependency injection functions: get_db (DB session),
get_redis (Redis connection), get_current_user (JWT verification), require_role (RBAC
checks), rate_limiter dependency
| | | — websockets.py     # Defines WebSocket endpoints and logic, e.g., WS
/payments/status/{session_id}
| |
| | — core/              # Core application settings, configurations, security utilities
| | — __init__.py        # Makes 'core' a Python package
| | — config.py          # Defines Pydantic Settings model, loads environment variables from
.env using python-dotenv
| | — security.py        # Utility functions for: PIN hashing/verification (passlib), JWT
creation/decoding (python-jose), TOTP generation/verification (pyotp), data
encryption/decryption (cryptography library with KMS keys)
| |
| | — crud/              # Database Create, Read, Update, Delete (CRUD) layer - interacts
directly with models
| | — __init__.py        # Makes 'crud' a Python package, exports CRUD objects
| | — base.py            # Defines a generic BaseCRUD class with common methods (get,
get_multi, create, update, delete)
| | — crud_user.py       # CRUD operations for User, UserSecurity models
| | — crud_kyc.py        # CRUD operations for KycSubmission model
| | — crud_wallet.py     # CRUD operations for Portfolio, Wallet, TokenBalance,
Transaction, BitcoinUtxo models
| | — crud_ledger.py     # CRUD operations for InternalLedger, DepositTransaction,
WithdrawalRequest models
| | — crud_payment.py    # CRUD operations for PaymentSession, PaymentTransaction,
SwapTransaction, FeeDistribution models
| | — crud_invoice.py    # CRUD operations for Invoice model
| | — crud_subscription.py # CRUD operations for Subscription, PullPaymentApproval
models
| | — crud_merchant.py   # CRUD operations for Merchant, MerchantKyc,
MerchantSettlement, SettlementDetail, MerchantEmployee, MerchantTerminal models

```

```

| | | └─ crud_staking.py  # CRUD operations for StakingPosition, InterestAccrual models
| | | └─ crud_vip.py      # CRUD operations for VipTier, VipBenefitsLog, TierHistory
models
| | | └─ crud_compliance.py # CRUD operations for TravelRuleRecord,
BlockchainScreening, SuspiciousActivity, ComplianceReport models
| | | └─ crud_market.py    # CRUD operations for PriceAlert, PriceSnapshot models
| | | └─ db/               # Database session management, base model definition, initial data
seeding
| | | └─ __init__.py       # Makes 'db' a Python package
| | | └─ base_class.py    # Defines the Base for SQLAlchemy declarative models, potentially
includes common columns like id, created_at, updated_at
| | | └─ session.py       # Configures SQLAlchemy async engine, SessionLocal factory, and
provides the get_db dependency function
| | | └─ init_db.py       # (Optional) Script to create initial database data (e.g., admin user,
default settings) - potentially run via CLI command
| | | └─ models/          # SQLAlchemy ORM database models (defines table structure,
columns, relationships)
| | | └─ __init__.py       # Makes 'models' a Python package, imports all models for easier
access via `from app.models import User` etc.
| | | └─ user.py          # Defines User, UserSecurity models
| | | └─ kyc.py           # Defines KycSubmission model
| | | └─ wallet.py        # Defines Portfolio, Wallet, TokenBalance, Transaction, BitcoinUtxo
models
| | | └─ ledger.py        # Defines InternalLedger, DepositTransaction, WithdrawalRequest
models
| | | └─ payment.py       # Defines PaymentSession, PaymentTransaction,
SwapTransaction, FeeDistribution models
| | | └─ invoice.py       # Defines Invoice model
| | | └─ subscription.py  # Defines Subscription (RecurringPayment),
PullPaymentApproval models
| | | └─ merchant.py      # Defines Merchant, MerchantKyc, MerchantSettlement,
SettlementDetail, MerchantEmployee, MerchantTerminal models
| | | └─ staking_vip.py   # Defines StakingPosition, InterestAccrual, VipTier,
VipBenefitsLog, TierHistory models
| | | └─ compliance.py    # Defines TravelRuleRecord, BlockchainScreening,
SuspiciousActivity, ComplianceReport, RegulatorySubmission, MsbLicense models
| | | └─ market.py        # Defines PriceAlert, PriceSnapshot models
| | | └─ smart_contract.py # Defines SmartContract model (for tracking deployed
contracts)
| | | └─ schemas/         # Pydantic schemas for data validation and serialization (API

```

request/response bodies, data transfer objects)

- `__init__.py` # Makes 'schemas' a Python package, imports/exports key schemas for easier access

- `user.py` # Defines User schemas (User, UserCreate, UserUpdate, UserInDB)

- `token.py` # Defines Token schemas (Token, TokenData for JWT)

- `security.py` # Defines schemas for security operations (PinChange, TotpEnableResponse, TotpVerifyRequest, WhitelistEntryCreate, etc.)

- `kyc.py` # Defines KycSubmission schemas (KycSubmissionCreate, KycSubmissionStatus)

- `wallet.py` # Defines Wallet schemas (Portfolio, Wallet, TokenBalance, Transaction, BalanceResponse, HistoryResponse, PortfolioCreate, WalletCreate)

- `transaction.py` # Defines Transaction schemas (TransactionPrepareRequest, TransactionPrepareResponse, TransactionBroadcastRequest, TransactionBroadcastResponse, TransactionStatusResponse)

- `exchange.py` # Defines Exchange schemas (InternalTransferRequest, DepositAddressResponse, WithdrawalRequest, LedgerEntry, InternalBalanceResponse)

- `payment.py` # Defines Payment schemas (PaymentSessionCreate, PaymentSessionResponse, PaymentExecutionRequest, PaymentStatusUpdate)

- `invoice.py` # Defines Invoice schemas (InvoiceCreate, InvoiceResponse)

- `subscription.py` # Defines Subscription schemas (SubscriptionCreate, SubscriptionResponse, PullApprovalCreate, PullApprovalResponse)

- `merchant.py` # Defines Merchant schemas (MerchantRegisterRequest, MerchantResponse, MerchantKycSubmit, DashboardResponse, SettlementRequest, SettlementResponse)

- `market.py` # Defines Market schemas (CoinMarketData, ChartDataResponse, PriceAlertCreate, PriceAlertResponse)

- `staking.py` # Defines Staking schemas (StakingOption, StakeRequest, UnstakeRequest, StakingPositionResponse, StakingHistoryResponse)

- `vip.py` # Defines VIP schemas (VipStatusResponse, VipBenefitsResponse)

- `nft.py` # Defines NFT schemas (NftResponse, NftTransferPrepareResponse)

- `defi.py` # Defines DeFi schemas (SwapQuoteRequest, SwapQuoteResponse, SwapPrepareRequest, SwapPrepareResponse)

- `services/` # Business logic layer - orchestrates CRUD operations, interacts with external APIs, blockchain networks

- `__init__.py` # Makes 'services' a Python package

- `blockchain/` # Blockchain-specific interaction logic, abstracting away library details

- `__init__.py` # Makes 'blockchain' a Python package

- `base_blockchain_service.py` # (Optional) Abstract base class for common blockchain operations

- `solana_service.py` # Functions for Solana: `get_balance`, `get_token_balance`,


```

get_transaction_history, build_transfer_tx, broadcast_tx, interact_with_payment_router
(Anchor)
| | | |— evm_service.py      # Functions for EVM chains (Base, Polygon, ETH):
get_balance, get_token_balance, get_tx_history, build_transfer_tx, broadcast_tx, estimate_gas,
interact_with_payment_router (Solidity/web3.py), handle_permit2/erc4337 if applicable
| | | |— bitcoin_service.py  # Functions for Bitcoin: get_balance (from UTXOs),
get_tx_history, build_transfer_tx (UTXO selection, RBF/CPFP logic), broadcast_tx,
handle_spv/timelocks if needed
| | | |— rpc_client.py       # Centralized client/wrapper for making RPC calls (handling
retries, failover logic based on health checks)
| | | |— kyc_service.py      # Logic for handling KYC submissions, uploading files
securely to S3, interacting with KYC provider API, updating user KYC status
| | | |— payment_service.py  # Core logic for creating/managing payment sessions
(Redis), verifying/executing QR/NFC payments, interacting with smart contracts for
settlement, handling swaps (via DEX service), calculating fees
| | | |— notification_service.py # Functions to send push notifications via FCM
(send_push_notification) and emails via SendGrid/SES (send_email)
| | | |— onramp_service.py   # Logic to handle webhooks from Transak/Ramp, credit
user internal ledger (via ledger_service)
| | | |— compliance_service.py # Functions to interact with Chainalysis API
(screen_address), logic for Travel Rule data collection and threshold checks
| | | |— market_data_service.py # Functions to fetch asset prices, market data, charts
from CoinGecko/CoinMarketCap, manages caching (Redis)
| | | |— staking_service.py   # Logic for interacting with Solana staking protocols,
calculating/distributing interest (via ledger_service)
| | | |— ledger_service.py    # Manages the internal double-entry ledger:
record_transaction (ensuring debits=credits), get_balance, get_history for internal accounts
| | | |— dex_aggregator_service.py # Functions to interact with Jupiter API (get_quote,
get_swap_tx) and 1inch API (get_quote, get_swap_data)
| | | |— merchant_service.py  # Logic specific to merchants: registration, KYC
processing, dashboard data aggregation, settlement processing
| | |
| | |— tasks/                # Celery task definitions (or functions for BackgroundTasks) for
asynchronous operations
| | | |— __init__.py          # Makes 'tasks' a Python package
| | | |— celery_app.py        # Celery application instance configuration
| | | |— sync_balances.py     # Celery task (@celery_app.task) to periodically query and
update on-chain balances for users
| | | |— process_withdrawals.py # Celery task to process withdrawal requests from the
queue (checks balances, interacts with blockchain_service)
| | | |— monitor_deposits.py  # Celery task/beat schedule to monitor company hot
wallets for incoming deposits, credits internal ledger
| | | |— process_subscriptions.py # Celery task/beat schedule to execute recurring

```

payments based on due dates

| | |— send_notifications.py # Celery task for sending push/email notifications asynchronously to avoid blocking API responses

| | |— compliance_tasks.py # Celery tasks for background compliance checks, report generation

| | |— utils/ # General utility functions, constants, enums, custom exceptions

| | |— __init__.py # Makes 'utils' a Python package

| | |— helpers.py # Miscellaneous helper functions (e.g., formatting timestamps, generating unique IDs)

| | |— constants.py # Application-wide constants (e.g., chain IDs, default slippage, KYC levels, RPC URLs from config)

| | |— enums.py # Enumerations (e.g., TransactionStatus, KYCStatus, ChainId, UserRole, TierLevel)

| | |— exceptions.py # Custom exception classes for specific error handling (e.g., InsufficientBalanceError, KycLevelError)

| |— alembic/ # Alembic database migration scripts and environment configuration

| |— versions/ # Contains individual, auto-generated migration script files (e.g., xxxxx_add_user_table.py)

| |— env.py # Alembic environment setup, points to DB models and database URL from config

|— smart_contracts/ # Smart contract source code, tests, deployment scripts (Potentially separate repo)

| |— solana/ # Anchor project for Solana PaymentRouter program

| | |— programs/

| | | |— payment_router/ # Source code (Rust - lib.rs defining structs, instructions like initialize, process_payment, withdraw_fees)

| | |— tests/ # Integration tests (TypeScript/JavaScript using Anchor client)

| | |— migrations/ # Deployment scripts (e.g., deploy.ts)

| | |— Anchor.toml # Anchor project configuration (program details, provider settings)

| |— evm/ # Hardhat/Foundry project for EVM PaymentRouter contract

| |— contracts/ # Solidity source code (.sol files - PaymentRouter.sol, interfaces, libraries)

| |— tests/ # Contract tests (JavaScript/TypeScript using ethers.js/Hardhat or Solidity using Foundry)

| |— scripts/ # Deployment and interaction scripts (e.g., deploy.js, interact.js)

| |— hardhat.config.js # Hardhat configuration (networks, compiler settings - or foundry.toml)

|— tests/ # Backend automated tests using pytest

|— __init__.py # Makes 'tests' a Python package

- |— conftest.py # Pytest fixtures (e.g., async_client, db_session, redis_conn, current_user_token, mock external services)
- |— api/ # Integration tests for API endpoints (testing request/response, auth, status codes)
 - |— v1/
 - |— test_auth.py
 - |— test_users.py
 - |— test_security.py
 - |— test_wallets.py
 - |— test_transactions.py
 - |— test_exchange.py
 - |— test_payments.py
 - |— test_invoices.py
 - |— test_subscriptions.py
 - |— test_merchants.py
 - |— test_market.py
 - |— test_staking.py
 - |— test_vip.py
 - |— test_nfts.py
 - |— test_defi.py
- |— crud/ # Unit tests for CRUD operations (mocking DB session/engine)
 - |— test_crud_user.py
 - |— test_crud_kyc.py
 - |— test_crud_wallet.py
 - |— ... (for each crud file)
- |— services/ # Unit tests for service layer logic (mocking CRUD, external APIs, blockchain calls)
 - |— test_solana_service.py
 - |— test_evm_service.py
 - |— test_bitcoin_service.py
 - |— test_kyc_service.py
 - |— test_payment_service.py
 - |— test_notification_service.py
 - |— test_onramp_service.py
 - |— test_compliance_service.py
 - |— test_market_data_service.py
 - |— test_staking_service.py
 - |— test_ledger_service.py
 - |— test_dex_aggregator_service.py
 - |— test_merchant_service.py
- |— utils/ # Unit tests for utility functions
 - |— test_helpers.py
 - |— test_security_utils.py # Tests for hashing, jwt, totp etc. in core.security

└─ test_exceptions.py

3.2. Frontend (/vextor7-mobile/)

/vextor7-mobile/

- └─ .env # Environment variables (API URL, third-party frontend keys - NOT COMMITTED)
- └─ .gitignore # Specifies intentionally untracked files that Git should ignore
- └─ app.json # Expo configuration file (name, icon, splash, permissions, plugins like react-native-nfc-manager, custom native code hooks for Trust Wallet Core)
- └─ babel.config.js # Babel configuration (presets for react-native, plugins like module-resolver for path aliases)
- └─ eas.json # Configuration for EAS Build and Submit (profiles for dev, preview, production; secrets management)
- └─ package.json # Project metadata, dependencies (react-native, expo, react-navigation, axios, ethers, @solana/web3.js, etc.), scripts
- └─ yarn.lock / package-lock.json # Exact versions of Node.js dependencies
- └─ tsconfig.json # TypeScript configuration (if using TypeScript)
- └─ README.md # This file
- └─ assets/ # Static assets (fonts, images, icons)
 - └─ fonts/ # Custom font files (.ttf, .otf)
 - └─ icons/ # App icon source file(s) (.png, .psd)
 - └─ images/ # Logos, illustrations, background images (.png, .jpg, .svg)
 - └─ splash/ # Splash screen image source file(s) (.png)
- └─ src/ # Main application source code directory
 - └─ api/ # Configuration and functions for interacting with the backend API
 - └─ index.ts # Configures Axios instance (sets base URL, default headers, request/response interceptors for JWT handling and error reporting)
 - └─ services/ # Typed functions for making specific API calls, grouped by feature
 - └─ authApi.ts # Functions: register, login, refreshToken, logout
 - └─ userApi.ts # Functions: getMe, updateMe, searchUser
 - └─ securityApi.ts # Functions: changePin, enableTotp, verifyTotp, disableTotp, getBackupCodes, submitKyc, getKycStatus, addWhitelist, getWhitelist, deleteWhitelist
 - └─ walletApi.ts # Functions: createPortfolio, getPortfoliosAndWallets, addWallet, deleteWallet, renamePortfolio, getBalance, getHistory
 - └─ transactionApi.ts # Functions: prepareTransaction, broadcastTransaction, getTransactionStatus, prepareCancelTransaction
 - └─ exchangeApi.ts # Functions: getInternalBalance, internalTransfer, getInternalTransfers, getDepositAddress, getDeposits, requestWithdrawal, getWithdrawals

- | |—— paymentApi.ts # Functions: createPaymentSession, getPaymentSession, executePayment
- | |—— invoiceApi.ts # Functions: createInvoice, getInvoices, getInvoiceDetails, payInvoice
- | |—— subscriptionApi.ts # Functions: createSubscription, getSubscriptions, updateSubscription, approvePullPayment, getPullApprovals, revokePullApproval
- | |—— merchantApi.ts # Functions: registerMerchant, getMerchantDetails, submitMerchantKyc, getDashboardData, getMerchantTransactions, requestSettlement, getSettlements, addEmployee, getEmployees
- | |—— marketApi.ts # Functions: getCoins, getCoinChart, createAlert, getAlerts, deleteAlert
- | |—— stakingApi.ts # Functions: getStakingOptions, stake, unstake, getStakingPositions, getStakingHistory
- | |—— vipApi.ts # Functions: getVipStatus, getVipBenefits
- | |—— nftApi.ts # Functions: getNfts, prepareNftTransfer
- | |—— defiApi.ts # Functions: getSwapQuote, prepareSwap
- | |—— components/ # Reusable UI components used across multiple screens
- | |—— common/ # Basic, generic UI elements without specific business logic
- | |—— Button.tsx # Customizable button component
- | |—— Input.tsx # Text input component with validation states
- | |—— Card.tsx # Generic card container
- | |—— Modal.tsx # Reusable modal component
- | |—— Loader.tsx # Loading indicator component
- | |—— Icon.tsx # Wrapper for icon library (e.g., Feather, MaterialIcons)
- | |—— Typography.tsx # Standardized Text components (Heading, Body, etc.)
- | |—— layout/ # Components defining screen structure and layout
- | |—— Container.tsx # Standard screen container with padding/margins
- | |—— Header.tsx # Screen header component (title, back button, actions)
- | |—— SafeAreaViewWrapper.tsx # Handles safe area insets for different devices
- | |—— specific/ # Components with logic tied to specific features
- | |—— WalletCard.tsx # Displays summary of a wallet's balance
- | |—— TransactionItem.tsx # Renders a single item in the transaction list
- | |—— NftCard.tsx # Displays an NFT thumbnail and basic info
- | |—— QrCodeDisplay.tsx # Displays a QR code (using react-native-qrcode-svg)
- | |—— PinInput.tsx # Component for entering a 6-digit PIN
- | |—— index.ts # Exports all components for easier importing via '@components'
- | |—— constants/ # Application-wide constant values, enums, configurations
- | |—— colors.ts # Defines the application color palette (primary, secondary, accent, etc.)
- | |—— dimensions.ts # Defines common spacing, font sizes, border radii
- | |—— apiRoutes.ts # Maps function names or keys to backend API endpoint paths

- | |— chains.ts # Configuration objects for supported blockchain networks (name, ID, RPC URL, explorer URL, native currency, icons)
- | |— kycLevels.ts # Defines KYC levels and their associated limits/requirements
- | |— index.ts # Exports all constants
- |— contexts/ # React Context API providers and hooks for global state management
 - | |— AuthContext.tsx # Manages user session (token, user info), login/logout functions
 - | |— WalletContext.tsx # Manages wallet state (portfolios, addresses, balances, transaction history), functions to refresh data
 - | |— SettingsContext.tsx # Manages user preferences (theme, language, primary currency)
 - | |— NetworkContext.tsx # Manages network status (online/offline), selected RPC endpoints, health checks
- |— hooks/ # Custom React hooks encapsulating reusable logic
 - | |— useAuth.ts # Hook to easily access AuthContext values and actions
 - | |— useWallets.ts # Hook to easily access WalletContext values and actions
 - | |— useBiometrics.ts # Hook abstracting platform-specific biometric checks (react-native-keychain or expo-local-authentication)
 - | |— useClipboard.ts # Hook for copying text to clipboard
 - | |— useDebounce.ts # Hook for debouncing input values
 - | |— useKycStatus.ts # Hook to fetch and manage KYC status display logic
- |— lib/ # Third-party library configurations or wrappers
 - | |— sentry.ts # Sentry initialization and configuration
 - | |— i18n.ts # Internationalization setup (e.g., using i18next)
- |— navigation/ # React Navigation setup (stacks, tabs, routing logic)
 - | |— index.tsx # Root navigator - checks auth status and renders AuthStack or MainTabNavigator
 - | |— types.ts # TypeScript types for navigation parameters
 - | |— AuthStack.tsx # Stack navigator for screens before login (Welcome, Login, Register, PINSetup, RecoverWallet)
 - | |— MainTabNavigator.tsx # Bottom tab navigator (Wallet, Swap, Payments, Market, Settings)
 - | | |— Stacks/ # Stack navigators nested within tabs for deeper navigation
 - | | | |— WalletStack.tsx # Screens accessible from the Wallet tab (WalletHome, AssetDetails, TxHistory, Receive, AddWallet, PortfolioSettings)
 - | | | |— SendStack.tsx # Screens for the Send flow (independent stack often presented modally)
 - | | | |— SwapStack.tsx # Screens for the Swap tab (SwapHome, SelectToken, SwapSettings, SwapConfirmation)
 - | | | |— PaymentStack.tsx # Screens for Payments tab (PaymentHome, ScanQR,

GenerateQR, NfcPay, InvoiceList, CreateInvoice)

| |— MarketStack.tsx # Screens for Market tab (MarketHome, CoinDetails, PriceAlerts)

| |— SettingsStack.tsx # Screens for Settings tab (SettingsHome, Profile, Security, Preferences, Networks, Support, About)

| |— screens/ # Components representing individual application screens, organized by feature/stack

| |— Auth/ # Authentication screens (LoginScreen, RegisterScreen, PinScreen, SeedPhraseScreen)

| |— Wallet/ # Wallet screens (WalletHomeScreen, AssetDetailsScreen, TransactionHistoryScreen, ReceiveScreen)

| |— SendReceive/ # Send flow screens (SendAmountScreen, SendAddressScreen, SendConfirmScreen)

| |— Swap/ # Swap screens (SwapScreen, SelectTokenScreen, SwapReviewScreen)

| |— Payments/ # Payment screens (ScanQrScreen, GenerateQrScreen, NfcScreen, InvoiceListScreen, InvoiceDetailsScreen)

| |— Staking/ # Staking screens (StakingOptionsScreen, StakeAmountScreen, StakingDashboardScreen)

| |— Market/ # Market screens (MarketListScreen, CoinDetailsScreen, PriceAlertListScreen, CreatePriceAlertScreen)

| |— Settings/ # Settings screens (SettingsScreen, ProfileScreen, SecuritySettingsScreen [2FA, PIN, Whitelist], PreferencesScreen)

| |— Merchant/ # Merchant screens (MerchantDashboardScreen, PosScreen, SettlementScreen, EmployeeManagementScreen)

| |— KYC/ # KYC flow screens (KycIntroScreen, KycLevel1Screen, DocumentUploadScreen, SelfieScreen)

| |— Onboarding/ # Initial app onboarding/wallet creation screens (WelcomeScreen, CreateOrImportScreen)

| |— services/ # Frontend-specific business logic, blockchain interactions (signing), native module interactions

| |— walletCoreService.ts # Wrapper providing a typed interface to Trust Wallet Core functions (createWallet, signTransaction, deriveAddress, etc.) - Requires Native Module Bridge

| |— solanaService.ts # Client-side Solana interactions using @solana/web3.js: signing transactions prepared by backend, interacting with WalletConnect/DApps

| |— evmService.ts # Client-side EVM interactions using ethers.js: signing transactions, interacting with WalletConnect/DApps, signing messages

| |— bitcoinService.ts # Client-side Bitcoin interactions using bitcoinjs-lib: signing transactions (UTXO management)

| |— notificationService.ts # Setup and handling of FCM push notifications (requesting

```

permission, receiving messages using expo-notifications or react-native-firebase)
|   |— nfcService.ts    # Setup and handling of NFC events using
react-native-nfc-manager (reading tags, HCE for Android)
|   |— biometricService.ts # Service layer for biometric authentication logic using hooks
|   |— clipboardService.ts # Service layer for clipboard interactions using hooks
|
|— store/                # Global state management if needed (e.g., Zustand or Redux Toolkit) -
Optional, Context might suffice
|   |— slices/           # State slices/reducers (e.g., uiSlice for managing global UI state like
loading indicators)
|   |— index.ts          # Store configuration and root reducer/selectors
|
|— styles/               # Global styles, theme configuration
|   |— theme.ts          # Defines theme properties (colors, fonts, spacing) used throughout
the app
|
|— types/                # Global TypeScript type definitions
|   |— index.ts          # Exports all types
|   |— navigation.ts     # Types specific to React Navigation parameters (if not in
navigation/types.ts)
|   |— api.ts            # Types for API request/response objects (ideally generated or
matching backend schemas)
|   |— wallet.ts         # Domain types (Wallet, Transaction, Token, NFT, etc.)
|
|— utils/                # Utility functions and helpers not specific to React/state
|   |— helpers.ts        # General utility functions (formatting numbers/dates, validating
input)
|   |— storage.ts        # Async wrapper functions for interacting with AsyncStorage (or
MMKV for performance)
|   |— crypto.ts         # Client-side cryptographic helpers (if needed beyond Wallet Core,
e.g., hashing for specific checks)
|   |— blockchain.ts     # Blockchain-related utility functions (address validation, unit
conversions, explorer links)
|
|— App.tsx               # Root application component: Sets up providers (Contexts,
Navigation, Theme), global error handling (Sentry), loads fonts/assets

```

(Note: The smart_contracts directory might be managed in a separate repository for better isolation and testing.)

4. Environment Setup

4.1. Prerequisites

- Python 3.9+ and Pip
- Poetry (Recommended for Python dependency management)
- Node.js (LTS version) and npm/yarn
- React Native development environment (Expo CLI, Android Studio/Xcode, Watchman, CocoaPods for iOS)
- Docker and Docker Compose (Recommended for local DB/Redis/Celery)
- PostgreSQL Client (e.g., psql)
- Redis Client (e.g., redis-cli)
- AWS Account & CLI (Configured for deployment and services like S3, KMS)
- Git

4.2. Backend Setup (FastAPI - /vextor7-backend/)

1. Clone the Repository:

```
git clone <repository-url>
cd vextor7-backend
```

2. Install Dependencies:

```
poetry install
```

3. Setup Environment Variables:

- Copy .env.example to .env (if example exists) or create .env.
- Populate .env with necessary configurations (Database URL, Redis URL, JWT Secret, RPC URLs, API Keys for external services, AWS credentials). Refer to app/core/config.py for required variables.
Example minimal .env
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/vextor7db #
Use asyncpg driver
REDIS_URL=redis://localhost:6379/0
SECRET_KEY=your_strong_secret_key_for_jwt # openssl rand -hex 32
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
SOLANA_RPC_URL=[https://api.mainnet-beta.solana.com](https://api.mainnet-beta.s
olana.com) # Replace with QuickNode/Alchemy later
BASE_RPC_URL=https://mainnet.base.org # Replace later
... other RPCs and API Keys ...

4. Setup Docker Services (DB, Cache, Broker):

```
docker-compose up -d postgres redis celery-broker # Adjust services based on
```

`docker-compose.yml`

5. Run Database Migrations:

`poetry run alembic upgrade head`

6. Run Celery Worker & Beat (if using Celery):

- Open separate terminals:

`poetry run celery -A app.tasks.celery_app worker --loglevel=info`

`poetry run celery -A app.tasks.celery_app beat --loglevel=info`

7. Run the Backend Server:

`poetry run uvicorn app.main:app --reload --host 0.0.0.0 --port 8000`

Access API docs at <http://localhost:8000/docs>.

4.3. Frontend Setup (React Native - /vextor7-mobile/)

1. Clone the Repository:

`git clone <repository-url>`

`cd vextor7-mobile`

2. Install Dependencies:

`yarn install`

`# or`

`# npm install`

3. Install iOS Pods:

`cd ios && pod install && cd ..`

4. Setup Environment Variables:

- Copy `.env.example` to `.env` (if example exists) or create `.env`.
- Add the backend API base URL and any necessary frontend API keys (Transak, Sentry, Firebase). Use your machine's local IP address, not localhost, if running on a physical device.
`API_BASE_URL=http://<your-local-ip-address>:8000/api/v1`
`# Example Frontend Keys`
`TRANSAK_API_KEY=...`
`SENTRY_DSN=...`

- Configure react-native-dotenv or equivalent to load these variables.

5. Integrate Trust Wallet Core:

- Follow the official Trust Wallet Core documentation for React Native integration. This usually involves:
 - Adding native dependencies to `android/build.gradle` and `ios/Podfile`.

- Configuring local.properties (Android) with GitHub credentials/PAT if required by TWC build process.
 - Writing native bridge modules (Java/Kotlin for Android, Objective-C/Swift for iOS) to expose TWC functions to JavaScript, or using pre-built community bridges if available and compatible.
 - Ensure necessary build tools (NDK for Android, specific Xcode version) are installed.
6. **Configure Native Modules:** Ensure other native modules (NFC, Biometrics, Keychain/Keystore) are correctly linked and configured (e.g., adding permissions in AndroidManifest.xml and Info.plist).
7. **Run the Frontend App:**
- Ensure the backend server is running.
 - **Using Expo:**
npx expo start
 - Scan QR code or use simulator (Press a or i).
 - **Using React Native CLI (if applicable):**
npx react-native run-android
or
npx react-native run-ios

5. Database Models Overview (PostgreSQL)

(This provides a high-level view. Refer to the specific files in /app/models/ for exact SQLAlchemy definitions, relationships, and constraints)

- **Core User & Security:** user.py (User, UserSecurity), kyc.py (KycSubmission), security.py (AddressWhitelist) - *Note: security models might be grouped differently*
- **Wallet:** wallet.py (Portfolio, Wallet, TokenBalance, Transaction, BitcoinUtxo)
- **Smart Contracts:** smart_contract.py (SmartContract)
- **Internal Ledger & Exchange:** ledger.py (InternalLedger, DepositTransaction, WithdrawalRequest), market.py (ExchangeRate - if storing rates)
- **Payments:** payment.py (PaymentSession, PaymentTransaction, SwapTransaction, FeeDistribution)
- **Invoices & Subscriptions:** invoice.py (Invoice), subscription.py (Subscription/RecurringPayment, PullPaymentApproval)
- **Merchant:** merchant.py (Merchant, MerchantKyc, MerchantSettlement, SettlementDetail, MerchantEmployee, MerchantTerminal)
- **Staking & VIP:** staking_vip.py (StakingPosition, InterestAccrual, VipTier, VipBenefitsLog, TierHistory)
- **Compliance:** compliance.py (TravelRuleRecord, BlockchainScreening, SuspiciousActivity, ComplianceReport, RegulatorySubmission, MsbLicense)

- **Market:** market.py (PriceAlert, PriceSnapshot)

6. API Endpoints Overview

Base URL: /api/v1

(This is a summary. Refer to the specific files in /app/api/v1/endpoints/ for detailed implementations including request/response schemas defined in /app/schemas/)

- **Auth (auth.py):** Register, Login, Refresh Token, Logout.
- **Users (users.py):** Get/Update Profile, Search User.
- **Security (security.py):** PIN Change, TOTP Mgmt, KYC Submit/Status, Whitelist Mgmt.
- **Wallets (wallets.py):** Portfolio Mgmt, Wallet Mgmt, Balance & History Queries.
- **Transactions (transactions.py):** Prepare/Broadcast On-chain Tx, Get Status, Prepare Cancel.
- **Exchange (exchange.py):** On-Ramp Webhook, Internal Balance/Transfer/History, Deposit Address/History, Withdrawal Request/History.
- **Payments (payments.py):** Payment Session Create/Get/Execute, WebSocket Status (websockets.py).
- **Invoices (invoices.py):** CRUD & Payment.
- **Subscriptions (subscriptions.py):** Subscription CRUD, Pull Payment Approval CRUD.
- **Merchants (merchants.py):** Register, KYC, Dashboard Data, Tx History, Settlement Mgmt, Employee Mgmt.
- **Market (market.py):** Coin List/Chart Data, Price Alert CRUD.
- **Staking (staking.py):** Options, Stake/Unstake, Positions, History.
- **VIP (vip.py):** Status, Benefits.
- **NFTs (nfts.py):** Get NFTs, Prepare Transfer.
- **DeFi (defi.py):** Swap Quote/Prepare.
- **Admin (admin.py):** User Mgmt, KYC Review, Tx Monitoring, Withdrawal Approval, Compliance Review, Merchant Approval, Settlement Processing.

7. Key Services & Libraries (Backend)

- **Web Framework:** FastAPI, Uvicorn (ASGI Server)
- **Data Validation:** Pydantic
- **Authentication:** python-jose[cryptography] (JWT), passlib[bcrypt] (Hashing)
- **2FA:** pyotp
- **Database ORM:** SQLAlchemy (asyncio support)
- **DB Driver:** asyncpg
- **Migrations:** Alembic
- **Caching/Queues:** redis (async version)
- **Background Tasks:** Celery, Flower (Monitoring)
- **Scheduling:** APScheduler (async version)
- **Blockchain Libs:** solana-py, web3.py, python-bitcoinlib (or others)
- **API Clients:** httpx (Async HTTP requests)

- **File Storage:** aioboto3 (Async AWS SDK for S3, KMS)
- **Email:** python-sendgrid or aioboto3 (for AWS SES)
- **Testing:** pytest, pytest-asyncio, httpx (for test client)

8. Running the Application

- **Backend:** Follow setup steps in 4.2. Ensure Docker services and Celery (if used) are running. Server runs on `http://localhost:8000`. API Docs at `/docs`.
- **Frontend:** Follow setup steps in 4.3. Ensure the backend is running and `API_BASE_URL` is correct. Start via `npx expo start` or `npx react-native run-*`.

9. Deployment

(Refer to previous detailed sections - involves containerization (Docker), IaC (AWS CDK/Terraform), CI/CD (GitHub Actions), AWS services (ECS/Fargate, RDS, ElastiCache, S3, KMS, ALB, etc.), EAS Build/Submit for mobile.)

10. Testing Strategies

(Refer to previous detailed sections - includes Unit Tests (Pytest, Jest), Integration Tests, E2E Tests (Detox/Maestro), Smart Contract Tests (Anchor, Hardhat/Foundry), Security Testing (SAST, DAST, Pen Testing, Audits), Performance Testing (Locust/k6), Usability Testing.)

11. Security Considerations

(Refer to previous detailed sections - emphasizes Non-Custodial nature, Secure Auth/Authz, Data Encryption (KMS), Input Validation, Rate Limiting, KYC/AML/Compliance (Chainalysis), Smart Contract Security (Audits), Secure Infrastructure, Dependency Scanning, Secure Client-Side Key Storage, Monitoring.)

12. Contribution Guidelines

(Refer to previous detailed sections - Gitflow Branching, Conventional Commits, Code Style (Linters/Formatters), Pull Requests (Reviews, CI Checks), Mandatory Testing, Documentation Updates.)